PROYECTOS EN TECNOLOGÍA

Git y GitHub

Geoffrey Hecht

MOTIVACIÓN


informe - Copy


informe - revisado


informe


informe_22_06_2022


informe_v1


informe_v2

Básicamente, evitar esto:

- ¿Cuál es la última versión del archivo?
- ¿En qué orden se crearon estas versiones?
- ¿En qué versiones trabajó mi colega?

SISTEMAS DE CONTROL DE VERSIONES

Un **VCS** (*Version Control System*, sistema de control de versiones) registra la historia de los archivos de un proyecto y permite volver a cualquier versión anterior.

Sistema	Año	Modelo
CVS (*Concurrent Versions System*)	1990	centralizado: un servidor guarda la historia
Subversion (SVN)	2000	centralizado
Git	2005	distribuido: cada copia guarda la historia completa
Mercurial	2005	distribuido

GIT Y GITHUB

- **Git**: el programa de control de versiones, instalado en cada equipo. Funciona sin conexión a internet
- **GitHub**: un servicio web que aloja repositorios remotos y agrega herramientas de colaboración, como la revisión de cambios y los tableros de tareas
- Servicios equivalentes a GitHub: GitLab, Bitbucket

Casi todos los comandos de git trabajan sobre el repositorio local. GitHub interviene solo al sincronizar: `clone`, `push` y `pull`.

VENTAJAS

- Gestión del código fuente, y de sus versiones, en proyectos de software
- Trabajo en colaboración: intercambios simples, trazabilidad, gestión de conflictos
- Copia de seguridad en uno o varios servidores en línea
- Historial del trabajo en cada archivo: ¿quién hizo qué, y cuándo?
- Posibilidad de volver a un estado previo, de un archivo o del proyecto completo
- Visualización de los cambios en el tiempo

GIT

Toda la historia de versiones y la configuración del proyecto están en una sola carpeta oculta, `.git`, en la raíz del proyecto. No hace falta conocer su contenido, solo no borrarla.

```
2026-G01/
├── .git/
├── .github/
├── data/
├── notebooks/
├── .gitignore
└── README.md
```

```
.git/
├── hooks/
├── info/
├── objects/
├── refs/
├── config
├── description
├── HEAD
└── index
```

CONCEPTOS: REPOSITORIO Y COMMIT

- **Repositorio** (*repository*): el almacenamiento de la historia del proyecto, contenido en el `.git`. Puede estar en uno o varios equipos, local o remoto
- **Commit** (o revisión): una versión instantánea del proyecto, como una foto. Tiene:
  - un identificador, el **hash**: una huella de 40 caracteres hexadecimales calculada a partir del contenido, por ejemplo `7f67902...`
  - un autor, una fecha y una descripción (el mensaje)
  - uno o varios commits **padres**, lo que forma la historia

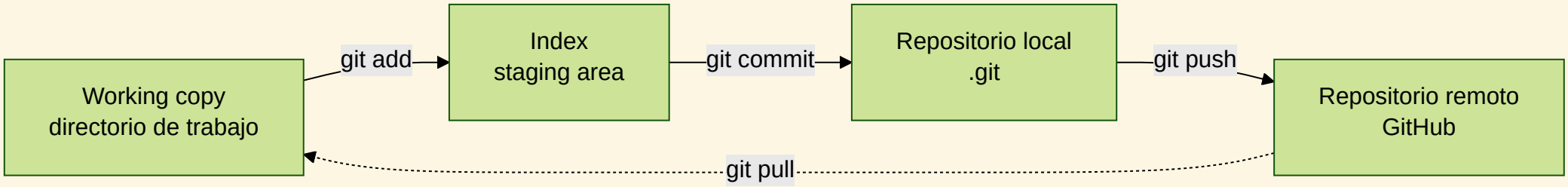
CONCEPTOS: COPIA DE TRABAJO E INDEX

- **Working copy** (copia de trabajo): los archivos tal como están en el disco. Puede ser diferente del último commit
- **Index** (o *staging area*, área de preparación): zona de paso entre la copia de trabajo y el repositorio. Un cambio entra en el próximo commit solo si fue agregado (*staged*) al index

```
$ git status
Changes not staged for commit:
        modified:   README.md

Untracked files:
        .gitignore
        notebooks/
```

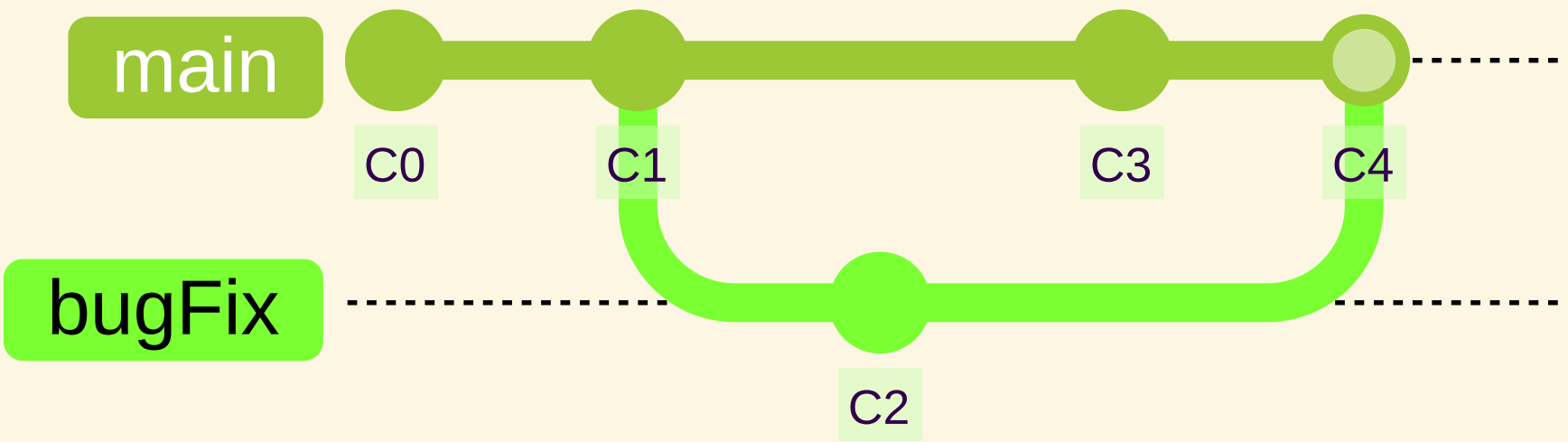
ZONAS DE GIT



- Un archivo modificado vive primero en la **copia de trabajo**
- `git add` lo pasa al **index**: ahí se decide qué entra en el próximo commit
- `git commit` lo graba en el **repositorio local**, y solo `git push` lo hace visible para el resto del equipo

CONCEPTOS: RAMAS

Una **rama** (*branch*) es una versión paralela del proyecto. La rama principal, por defecto, se llama `main` (o `master` en repositorios antiguos).

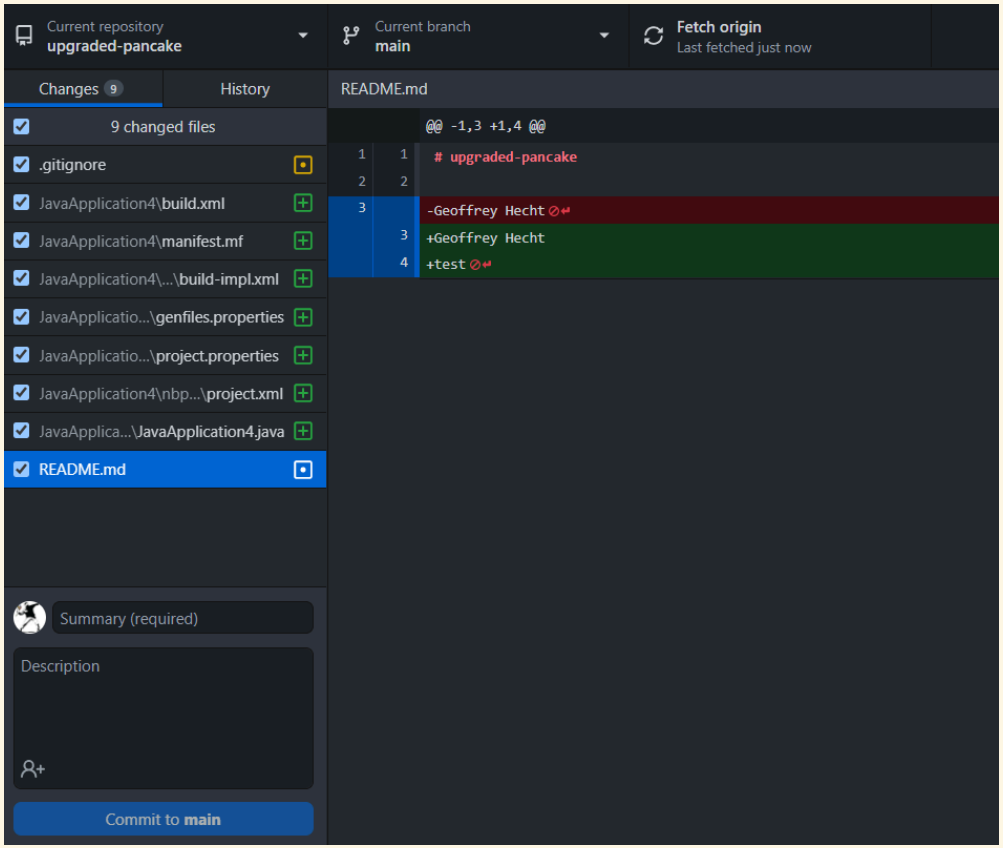


`C2` y `C3` se hicieron en paralelo a partir de `C1`; el commit `C4` fusiona las dos ramas y tiene, por lo tanto, dos padres.

CONCEPTOS: CLIENTE

Un **cliente** es la herramienta para usar git: la línea de comandos o una interfaz gráfica.

- En este curso usaremos la **línea de comandos**
- Interfaces gráficas: GitHub Desktop, GitKraken, las integradas en VS Code o IntelliJ
- Lista: git-scm.com/downloads/guis



GitHub Desktop

TERMINAL

La **terminal** (o línea de comandos) es un programa donde se escriben órdenes en texto. En Windows, Git instala una llamada *Git Bash*; en macOS y Linux viene incluida.

Comando	Qué hace
`pwd`	Muestra la carpeta actual (*print working directory*)
`ls`	Lista los archivos de la carpeta actual
`cd <carpeta>`	Entra en una carpeta
`cd ..`	Sube a la carpeta que contiene la actual
`mkdir <carpeta>`	Crea una carpeta

LECTURA DE LOS EJEMPLOS

```
$ pwd
/home/usuario/proyectos
$ git status
fatal: not a git repository (or any of the parent directories): .git
```

- Una línea que empieza con `$` es un comando que se escribe, sin el `$`; las demás son la respuesta
- **Resaltado**: la parte que hay que leer. En gris: detalles que se pueden omitir. `[...]`: salida recortada
- `<carpeta>`, `<rama>`: un valor que se reemplaza, sin los signos `<` y `>`

El error del ejemplo aparece al usar git fuera de un repositorio: hay que entrar con `cd` a la carpeta que contiene el `.git`.

CONFIGURACIÓN INICIAL

Una sola vez por equipo, antes del primer commit:

```
$ git config --global user.name "Nombre Apellido"
$ git config --global user.email "nombre.apellido@udec.cl"
```

- Cada commit registra este nombre y este correo como autor
- Con el mismo correo que la cuenta de GitHub, los commits quedan asociados a esa cuenta
- `--global`: la configuración vale para todos los repositorios del equipo

AUTENTICACIÓN EN GITHUB

GitHub no acepta la contraseña de la cuenta en la terminal: la pide, pero el acceso falla.

```
$ git push
Username for 'https://github.com': usuario
Password for 'https://usuario@github.com':
remote: Support for password authentication was removed on August 13, 2021.
[...]
fatal: Authentication failed for 'https://github.com/[...]'
```

- **Windows**: el gestor de credenciales de Git abre el navegador para iniciar sesión la primera vez
- **GitHub CLI** (*command line interface*, interfaz de línea de comandos): `gh auth login` configura el acceso con un asistente
- **Token personal** (*personal access token*): una clave generada en GitHub, que se escribe en lugar de la contraseña
- **Clave SSH** (*Secure Shell*): la clave pública se registra en GitHub y el remoto usa una URL `git@github.com:...`

GIT INIT

Crea un repositorio local vacío en la carpeta actual.

```
$ mkdir newproject
$ cd newproject/
$ git init
Initialized empty Git repository in ~/proyectos/newproject/.git/
```

GIT CLONE

Crea una copia local de un repositorio remoto, con toda su historia.

```
$ git clone https://github.com/TECH-MACI/2026-G01.git
Cloning into '2026-G01'...
remote: Enumerating objects: 31, done.
remote: Counting objects: 100% (31/31), done.
remote: Compressing objects: 100% (24/24), done.
remote: Total 31 (delta 3), reused 25 (delta 2), pack-reused 0
Receiving objects: 100% (31/31), 9.84 MiB | 4.12 MiB/s, done.
Resolving deltas: 100% (3/3), done.
```

Resultado: una carpeta `2026-G01/` que ya contiene su `.git`, conectada al remoto, llamado `origin` por defecto. Como el repositorio del grupo es privado, `clone` pide autenticarse.

GIT STATUS

Muestra el estado de la copia de trabajo y del index: qué cambió y qué entrará en el próximo commit.

```
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        notebooks/
```

GIT DIFF

Muestra las líneas modificadas en la copia de trabajo que todavía no pasan al index. `git diff --staged` muestra las que ya están en el index.

```
$ git diff
diff --git a/README.md b/README.md
index 5c1e8a2..9f04b7d 100644
--- a/README.md
+++ b/README.md
@@ -1,3 +1,5 @@
+# 2026-G01
+
 ### About

 Este repositorio contiene informacion publica sobre [...]
```

Las líneas que empiezan con `+` se agregaron y las que empiezan con `-` se eliminaron; las demás se muestran como contexto.

GIT ADD

Agrega al index un archivo nuevo o modificado. `git add .` agrega todos los archivos de la carpeta y sus subcarpetas.

```
$ git add README.md
$ git add .
$ git status
[...]
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   .gitignore
        modified:   README.md
        new file:   notebooks/01_exploracion.ipynb
        new file:   notebooks/02_limpieza.ipynb
```

.GITIGNORE

El archivo `.gitignore`, en la raíz del repositorio, lista los archivos que git ignora: no aparecen en `git status` y `git add .` no los agrega.

```
# sistema y entornos
.DS_Store
.venv/
__pycache__/
.ipynb_checkpoints/

# datos generados y secretos
data/procesados/
.env
```

- GitHub rechaza los archivos de más de 100 MB
- Un secreto (clave, contraseña) versionado queda en la historia, aunque se borre después
- `.gitignore` no afecta a los archivos que ya están versionados

El año pasado, 3 de los 4 repositorios de la organización no tenían `.gitignore`, y había `.DS_Store` (metadatos de macOS) y `.ipynb_checkpoints/` versionados.

GIT RM

Elimina un archivo y registra la eliminación en el index, para el próximo commit. Con `--cached`, el archivo deja de estar versionado pero sigue en el disco.

```
$ git rm notebooks/prueba.ipynb
rm 'notebooks/prueba.ipynb'
$ git rm --cached .DS_Store
rm '.DS_Store'
$ git status
[...]
Changes to be committed:
        deleted:    .DS_Store
        deleted:    notebooks/prueba.ipynb

Untracked files:
        .DS_Store
```

`.DS_Store` sigue en la carpeta, ahora como archivo no versionado: agregado al `.gitignore`, git deja de mostrarlo.

GIT COMMIT

Crea un commit con el contenido del index. Siempre lleva un mensaje, después de `-m`.

```
$ git commit -m "feat: notebooks de exploración y limpieza de sned"
[main 7f67902] feat: notebooks de exploración y limpieza de sned
 4 files changed, 412 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 notebooks/01_exploracion.ipynb
 create mode 100644 notebooks/02_limpieza.ipynb
```

`main` es la rama que recibió el commit y `7f67902` son los primeros caracteres del hash del nuevo commit.

GIT LOG

Muestra la historia de commits, del más reciente al más antiguo. Con `--oneline`, una línea por commit.

```
$ git log --oneline
7f67902 (HEAD -> main) feat: notebooks de exploración y limpieza de sned
586e925 (origin/main, origin/HEAD) Initial commit
```

`HEAD` señala el commit en que está la copia de trabajo, aquí el último de `main`. `origin/main` es la última posición conocida de `main` en el remoto: sigue en `586e925`, porque el nuevo commit todavía no se envió.

GIT PUSH

Envía los commits locales al repositorio remoto. Conviene hacerlo con regularidad: mientras no se hace, el trabajo solo existe en su equipo.

```
$ git push
Enumerating objects: 18, done.
Counting objects: 100% (18/18), done.
Delta compression using up to 12 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (15/15), 15.81 KiB | 5.27 MiB/s, done.
Total 15 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/TECH-MACI/2026-G01.git
   586e925..7f67902  main -> main
```

La última línea resume el envío: la rama `main` del remoto pasó del commit `586e925` al `7f67902`.

GIT PULL

Recupera los commits del repositorio remoto y los integra en la rama local. Puede producir conflictos, que se resuelven a mano.

```
$ git pull
Already up to date.

$ git pull
[...]
From https://github.com/TECH-MACI/2026-G01
   7f67902..2f2a02d  main       -> origin/main
Updating 7f67902..2f2a02d
Fast-forward
 README.md | 1 -
 1 file changed, 1 deletion(-)
```

El primer `pull` no encuentra nada nuevo. El segundo trae un commit: la rama `main` local avanza de `7f67902` a `2f2a02d`, y el resumen indica que `README.md` perdió una línea.

CICLO DE TRABAJO

```
$ git pull                      # al empezar: traer el trabajo del equipo
  (editar archivos)
$ git status                    # revisar qué cambió
$ git add <archivos>
$ git commit -m "mensaje"
$ git push                      # al terminar: compartir los commits
```

- Hacer `pull` antes de empezar reduce los conflictos: se trabaja sobre la versión más reciente
- Varios commits pequeños, cada uno con un solo objetivo, son más fáciles de revisar y de deshacer que uno grande

CONFLICTOS

- Git resuelve solo muchos casos de cambios simultáneos: líneas distintas de un archivo, o funciones agregadas en lugares diferentes
- Cuando dos personas modifican **las mismas líneas**, git no puede elegir y pide resolverlo a mano
- Aun cuando la fusión es automática, conviene revisar que el resultado siga siendo lógico
- Una vez resuelto, se registra una **fusión** (*merge*)

CONFLICTO PASO A PASO

Un colega hizo push antes que usted: el remoto rechaza su push.

```
$ git push
To https://github.com/TECH-MACI/2026-G01.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to [...]
hint: Updates were rejected because the remote contains work that you do
hint: not have locally. [...] You may want to first integrate
hint: the remote changes (e.g., 'git pull ...') before pushing again.
```

Al integrar los cambios remotos, los dos cambios tocan las mismas líneas de `README.md`:

```
$ git pull
[...]
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
```

MARCADORES DE CONFLICTO

Git escribe las dos versiones dentro del archivo:

```
# 2026-G01

<<<<<<< HEAD
Prioridad por establecimiento
=======
Prioridad por comuna
>>>>>>> f38df92
```

- **HEAD** apunta a su último commit: entre `<<<<<<<` y `=======` está su versión
- Entre `=======` y `>>>>>>>` está la versión del último commit remoto, identificado por su hash
- Se edita el archivo para dejar la versión correcta, **sin los marcadores**

CERRAR EL CONFLICTO

Una vez editado el archivo, se agrega y se confirma. El mensaje de la fusión es automático, no hace falta `-m`:

```
$ git add README.md
$ git commit
[main ee1e06f] Merge branch 'main' of https://github.com/TECH-MACI/2026-G01
```

Si la fusión se complica, `git merge --abort` devuelve el repositorio al estado anterior al `pull`.

GIT BRANCH Y GIT CHECKOUT

- `git branch <nombre>` crea una rama, por ejemplo para trabajar en una nueva funcionalidad
- `git checkout <rama>` cambia la rama de trabajo local; también acepta un commit

```
$ git branch limpieza-sned
$ git checkout limpieza-sned
Switched to branch 'limpieza-sned'
$ git status
On branch limpieza-sned
nothing to commit, working tree clean
```

PRIMER PUSH DE UNA RAMA

El remoto todavía no conoce la rama: el primer push debe indicar a qué rama remota corresponde (su *upstream*).

```
$ git push
fatal: The current branch limpieza-sned has no upstream branch.
To push the current branch and set the remote as upstream, use

    git push --set-upstream origin limpieza-sned

$ git push --set-upstream origin limpieza-sned
[...]
 * [new branch]      limpieza-sned -> limpieza-sned
branch 'limpieza-sned' set up to track 'origin/limpieza-sned'.
```

GIT MERGE

`git merge <rama>` fusiona esa rama en la rama actual. Se ejecuta **desde la rama que recibe** los cambios, usualmente la principal.

```
$ git commit -m "fix: elimina duplicados en sned"   # en limpieza-sned
[limpieza-sned fd3a791] fix: elimina duplicados en sned
$ git checkout main
Switched to branch 'main'
$ git merge limpieza-sned
Updating ee1e06f..fd3a791
Fast-forward
 notebooks/02_limpieza.ipynb | 14 ++++++-------
 1 file changed, 7 insertions(+), 7 deletions(-)
```

Fast-forward: como `main` no avanzó desde que se creó `limpieza-sned`, git solo mueve el puntero de `main` al último commit de `limpieza-sned`, sin crear un commit de fusión.

MENSAJES DE COMMIT

Un buen mensaje es **informativo** y explica el **motivo** del commit; el detalle de lo que cambió ya lo muestra el historial (`git log`). Las etiquetas de Conventional Commits indican el tipo de cambio.

Buenos ejemplos

```
feat: implementa el sistema de compra de bebidas
fix: ajusta hiperparámetros de ... para reducir overfitting
docs: actualiza la documentación de la clase Comprador
viz: añade heatmap para identificar patrones regionales
data: remueve duplicados para evitar doble conteo de matrícula

Corrige detección de outliers en datos de matrícula
Estandariza formato de coordenadas entre datasets
```

Malos ejemplos (pueden costar puntos)

```
Actualización de los datos
Commit 5
Clase Bebida
No entiendo lo que está pasando con Git
aaaaa
El profe es bacán
```

EN EL PROYECTO

- Cada grupo trabaja en su repositorio `TECH-MACI/2026-GXX`, creado desde la plantilla `PROJECT-BASE`, que trae los datos y las plantillas de issues
- El acceso al repositorio se da por el equipo (*team*) del grupo en la organización
- La versión a evaluar debe estar en la rama `main`, en el último commit: no olviden hacer el merge
- Eviten subir archivos o hacer commits desde la interfaz web de GitHub: es una mala práctica y es limitada
- Pueden usar una interfaz gráfica (IntelliJ, VS Code, GitHub Desktop, GitKraken), pero se recomienda empezar por la línea de comandos para entender cada paso

RESUMEN DE COMANDOS

Comando	Qué hace
`git config --global`	Define el nombre y el correo del autor de los commits
`git init`	Crea un repositorio vacío en la carpeta actual
`git clone <url>`	Copia un repositorio remoto, con su historia
`git status`	Muestra el estado de la copia de trabajo y del index
`git diff`	Muestra los cambios que todavía no están en el index
`git add <archivo>`	Agrega cambios al index
`git rm <archivo>`	Elimina un archivo y registra la eliminación en el index
`git commit -m "mensaje"`	Crea un commit con el contenido del index
`git log --oneline`	Muestra la historia de commits
`git push`	Envía los commits locales al remoto
`git pull`	Recupera e integra los commits del remoto
`git branch <rama>`	Crea una rama
`git checkout <rama>`	Cambia de rama
`git merge <rama>`	Fusiona una rama en la rama actual

DOCUMENTACIÓN Y TUTORIALES

- git-scm.com/docs: documentación oficial
- GitHub Docs: inicio rápido
- learngitbranching.js.org: ejercicios interactivos, útiles para entender las ramas
- Software Carpentry: control de versiones con Git, en español, episodios 1 a 9